
Pic-AI-sso, Because More is More

Adam Fredriksson
adamfre@kth.se

Astrid Frykman
afrykman@kth.se

Jacob Westergren
jawester@kth.se

Zeshen Bao
zeshen@kth.se

Abstract

Diffusion models have become a popular choice for image generation in recent years. Transformers have increasingly replaced the classic U-Net architecture. In this work, we implemented a class-conditional diffusion model with a transformer-like U-Net architecture incorporating self- and cross-attention mechanisms. The model was tested on MNIST, CIFAR-10, and LFWPeople combined with CelebA datasets. Compared to a baseline Convolutional Variational Autoencoder, our model showed significant improvements in image generation, with all generated images receiving high subjective evaluations. The FID scores obtained for our Exponential Moving Average models were 144 and 218 for CIFAR-10 and the CelebA model pre-trained on LFWPeople, respectively, while our regular models achieved similar results with FID scores of 143 and 217. This indicates that while our model can successfully generate images, more training is needed to reach state-of-the-art performance. The code is available at [this https URL](#).

1 Introduction

Diffusion models are state-of-the-art models in image generation. They are widely used due to both the ability to generate high-quality images and because they are typically less tedious to train than, for instance, GANs. A high-level overview of the typical approach in modern diffusion models is that the models are trained to predict noise in an image in a sequence of time steps. By removing the predicted noise at each time step, the final image should be something similar to the images in the data set. This is called a Denoising Diffusion Probabilistic Model (DDPM) and was introduced in [1], whereas regular diffusion models were introduced in [2].

More specifically, a noise schedule is defined as the process that adds noise to an image x_0 in time steps such that at the final time step T , the image x_T is essentially pure noise described by a standard Gaussian distribution $\mathcal{N}(0, I)$. This process is sometimes viewed as a fixed forward pass of a variational autoencoder (VAE), or as a Markov process as the next time step only depends on the current step. This means that the distribution of the next step's image is conditioned only on the current time step's image. [3]

Moreover, this generative ability is relevant in areas other than the well-known AI art use cases and image editing. One example is in healthcare, where generative models can be used to generate synthetic medical data, which can significantly speed up the advancement speeds in many medical areas as medical data is often scarce[4].

In this work, we implemented a DDPM that most closely follows the original paper [1], but with certain improvements from [5], [6] and [7]. Inspired by transformers, we also implemented multi-head cross attention (MHCA). To the best of our knowledge, this is new in the application of DDPMs but has been done in earlier work such as [8]. We found that without architectural changes, the model could learn both class-conditioned and unconditioned sampling. Image quality is not as high as in the cited works but given the comparatively few number of iterations, it seems capable of learning to generate high-quality images. Given its popularity and proficiency in medical imaging and image segmentation, future work could, for instance, apply our architecture to the generation of molecular structures.

2 Related Work

The updated improvement of DDPM [5], included several incremental improvements to [1] that we did not use in our model, such as improved log-likelihood, cosine noise scheduler, and quicker sampling speed. Similarly, other improvements were suggested in [7]. While we did implement the additional attention heads at multiple levels as they did, we did not, for example, include scaled residual connections or adaptive group normalization. We also used classifier-free guidance, from [6], instead of classifier guidance used in [7]. This will be explained in more detail in section 1.

The concept of latent diffusion models, as described by [9], involves the integration of a U-Net architecture within a VAE, aiming to operate within a reduced dimensional space. This would be, however, unsuitable for the lower-resolution images that we considered.

Further progress has been made in the field of image generation since the DDPM papers were published. For instance, the main image generation model used in Stable Diffusion[10] is based on the scalability of Diffusion Transformers (DiTs), introduced in[11], which, in principle, replaces the U-Net with a visual transformer (ViT) [12]. Although their DiT-XL/2 model outperformed all previous diffusion models using a U-net, it is computationally significantly more expensive and requires a substantially larger dataset such as ImageNet[13]. Therefore, given limited resources, this project focused on improving a PixelCNN++[14] based U-net to adopt DiT’s features, such as MHCA and multi-head self-attention (MHSA).

3 Data

In this project, we trained and evaluated our model on four different datasets with different complexities, allowing us to gradually test our model in more complex scenarios. These four datasets, in ascending computational complexity order, were MNIST, CIFAR10, LFWPeople, and CelebA.

The initial dataset was the well-known MNIST dataset [15], containing 70,000 labelled images of hand-drawn digits in 28x28 grayscale image resolution. However, this data set was upsampled to be 32x32 to accommodate the four resolution levels in the DDPM, which will be described in section 4. This simple dataset served as our control data set, allowing us to quickly see if our model had the potential to produce reasonable results.

Following the MNIST dataset, we trained our model on the more complex CIFAR10 dataset [16], labelled 60 000 labelled images of 10 classes ranging from dogs to planes, all of which have a pixel resolution of 32x32 in RGB format. This increase in data complexity allowed us to evaluate how our model could handle more complex and meaningful generative tasks.

Afterwards, we used the LFWPeople [17] dataset, consisting of 13,000 labelled images of human faces, to pre-train the final model. Finally, the CelebA[18] dataset, consisting of 200K celebrity images, was used as a larger data set to continue training the pre-trained LFWPeople model. The images of both the facial datasets were re-scaled to a 64x64 from 250X250 and 256x256 pixel resolution for LFWPeople and CelebA respectively. These datasets were chosen for two reasons: First, it allowed us to test our model on higher-resolution images. Second, it enabled a simple qualitative evaluation, as human faces are easy for the human eye to recognize.

4 Methods

4.1 Components of Diffusion Model

The main components of the diffusion model, besides the actual model architecture, consist of a noise schedule that describes the process and parameters for adding noise to the images during training and then a sampling procedure as a denoising process. The sampling procedure is a bit more involved but essentially performs the forward pass in reverse. The model predicts the total noise in the image, of which some is removed from the image. Some scaled noise is also added to the image, before moving to the next time step in the reverse process. At the end, no scaled noise is added and the final image x_1 should be as if drawn from the data set’s distribution. More details can be found in Appendix A, and a complete derivation may be found in [3].

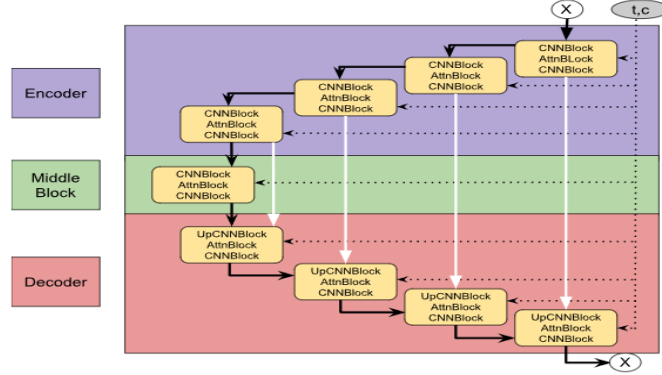


Figure 1: Simplified schematic of our model, where the different coloured areas showcase the encoder, middle block, and decoder. The black arrows represent the variable X 's path, the white arrows indicate the U-nets residual connections, and the dotted arrows show the time and class conditioning variables' paths. The naming in the figure is coherent with the naming in our code implementation.

The noise schedule is defined by the parameter β_t which, at a certain time step t , describes the variance of the distribution while the mean is given by $\sqrt{1 - \beta_t}x_{t-1}$ and therefore depends on the earlier time step. This is used to sub-sequentially add noise to an image until pure noise is reached. More details regarding this schedule can be found in [1] or in Appendix A. Both the linear schedule from [1] and the cosine schedule from [5] were implemented, but due to computational limitations, the linear schedule was ultimately used. This means that β ranged from 10^{-4} to 0.02 and that the number of time steps was 1000.

4.2 Models

Several models were implemented, gradually increasing the complexity of the models. As a baseline for the results, a Convolutional Variational Autoencoder (CVAE) was implemented. While the baseline itself was also implemented from scratch, it was based on a VAE architecture done in another course that we upgraded to a CVAE using inspiration from online sources such as [19]. The diffusion models implemented were all DDPMs with fixed variance, as in [1].

4.2.1 Model Architecture

The final model is a modified U-Net architecture, introduced in [20], that has both MHSA and MHCA. A simplified schematic of this architecture is shown in figure 1, showing the core components of our model.

As can be seen in figure 1, the U-Net architecture consists of a downward pass (encoder) and an upward pass (decoder), where the image goes through multiple resolution levels as it passes through the model. Each resolution level employs the use of two residual convolutional blocks, where each block consists of the Swift activation function, also called SiLU, group normalization [21], and two convolutional layers. The residual connection is then around the block, optionally applying a convolution 1x1 kernel convolution to the residual to get the correct channel value. Then, between the two residual blocks, self-attention was used, similar to the original DDPM paper [1]. However, as in the improved versions of DDPM [5] and [7], self-attention was implemented at all resolution levels instead of only at the 16×16 resolution level, in [1]. Moreover, MHSA was used with four attention heads at each attention block.

During the downward pass, the image is downsampled to half the resolution size at the end of each resolution level, with a copy of the image being stored before the downsampling occurs at each level. This process proceeds until the image reaches the bottom of the U-Net, called the bottleneck or the middle block. Then, the image will enter the upward pass, where the image will be upsampled to twice the resolution at the end of each level. The major difference is that the stored copies from the downpass will act as skip connections as they will be concatenated channel-wise to the upsampled image at the start of each new resolution level, as indicated by the white arrows in Fig 1.

Each resolution level also maintains a constant image size using padding and constant channel size. In the cases of CIFAR10, CelebA and LFWPeople, this was [64, 128, 256, 512] for the downpass and the reverse in the upward pass; meanwhile, for MNIST, the channels were [32, 64, 128, 256], reducing the model size substantially to just under 8 million parameters as compared to the 28.6 million the other two datasets used.

To give the model information about the current time step of the noise schedule, the time must be passed as an input to the model. This was done by first embedding the time using the sinusoidal positional embedding, defined in [22]. This is then passed through a simple Multi-Layer Perceptron with the SiLU activation function, before finally adding it at the second convolutional layer in each residual block, as shown by the dotted arrows in figure 1. This implementation was partly inspired by [23], which followed the exact definition given in [22]. Similarly, class information was passed to the model, although using an embedding layer with learnable parameters instead.

To further improve the model, classifier-free guidance was used, introduced in [6]. In short, this involves randomly dropping the class label during training, essentially training unconditionally for that batch. The probability of dropping the class label was set to 10%, as recommended in [3]. Then, when sampling a new image, a linear interpolation between unconditional output and class conditional outputs from the model is used to predict noise, with the interpolation weight w being specified in section 5. A weight of zero results results in unguided interpolation which unconditional sampling should use, while a stronger weight w applies stronger guidance and emphasis on the label. The implementation of classifier free guidance was inspired by the Github repository [24], which also was the source for the self-attention blocks, though our final code of this in our repository has been heavily modified.

The main novelty, to the best of our knowledge, of our approach is to use cross-attention between the residuals stored from the downpass of the U-Net and the upsampled elements. Typically, these are concatenated along the channel dimension, but we also calculated the cross-attention at every resolution level. This allows the model to learn which image regions it should focus on when upsampling, thereby enhancing the skip connections by combining spatial information from the encoder with semantic context from the decoder [8].

To stabilize training and improve performance, a dropout rate of 0.1 and an Exponential Moving Average (EMA) of model updates with a decay factor of 0.9999 were used, based on parameters reported in [1]. When using EMA, one essentially gets two copies of the model. One copy is trained using the optimizer as usual, while the other is updated using EMA. The non-EMA model is called the regular model, non-EMA or just DDPM in the remainder of the work.

4.3 Evaluation

To evaluate our results, we used both qualitative and quantitative methods. For comparing our model to our baseline, we used qualitative evaluation criteria as these were judged by our subjective opinion on the quality of the generated images.

For quantitatively comparing our model against other state-of-the-art models, we used the standard benchmark Frechet Inception Distance (FID) metric. This common metric allows us to easily compare our results against previous studies. For simplicity’s sake, we used the in-built FID computation function available in the `torchval` package [25]. We sent in batch pairs of real and generated images and let the function compute the distance between them.

5 Experiments

5.1 Training and Sampling Details

The AdamW optimizer, introduced in [26], from the PyTorch library [25] was utilized with an exponentially decaying learning rate with a starting value of 3×10^{-4} and the decay parameter γ was set to $\gamma = 0.9999$. The advantage of AdamW is that it is a variant of Adam that decouples the weight decay from the gradient update, providing more effective regularization and improved convergence properties. Towards the later stages of training, the decaying learning rate was substituted with a cosine annealing learning rate to avoid a too-small learning rate. This meant an oscillating learning rate between the starting value and zero.[26]

As suggested in [1] as well as in [3], a simple Mean Squared Error (MSE) loss between the predicted noise and the sampled noise was used as a loss function. To be completely mathematically stringent, the loss should be scaled depending on the values of the noise parameters for the current time step, but Ho et. al. [1] empirically found it to be better to omit this factor.

The training of the model for MNIST and CIFAR10 was done locally on a NVIDIA GTX 1070 graphics card, with batch sizes of 32 and 24 respectively. In both cases, the speed was roughly 3 iterations per second. The sampling time with 1000 time steps was around 90 seconds to generate 10 images. The MNIST model was trained for 5 epochs, and the CIFAR10 model was trained for 100 epochs.

The training of the model for the LFWPeople and CelebA were performed non-locally on Google Colab with a L4 GPU and A100 with a small batch size of 8-15 due to lack of memory. The limited amount of compute units per day limited the number of epochs that the model could train on with the given time constraints. If compared to [1], around 500k steps were run on a similar but larger dataset to LFWPeople to get generated images whereas our model only trained for roughly 93k steps. When the model was trained further on CelebA only another 90K steps were performed, totalling to less than half of the 0.5M steps taken by [1] on their CelebA model.

5.2 Results and Discussion

The qualitative comparison on MNIST between the baseline CVAE and our model is shown in figure 2, where the CVAE trained for 50 epochs and our model trained for 5 epochs. It should, however, be noted that while our model trained for fewer epochs, each epoch took much longer to complete. The class conditioning in our model is useful in labelled datasets such as MNIST, as it allows us to choose the image class to be generated. Other than that, the image quality is quite comparable with the baseline almost being the better choice simply due to reduced model size and sampling time.



Figure 2: Generated images for MNIST, with our DDPM, using class conditioning and guidance weight $w = 1$, to the left and the CVAE baseline to the right.

When moving on to the more complex dataset CIFAR10, the added expressiveness of our model is quite clearly beneficial. The main qualitative difference is the increased sharpness of the DDPM, as all of the CVAE’s images may resemble certain objects but are extremely blurry. Additionally, our model once again successfully generated images conditioned on class labels, although the actual image is not always completely logical or perfect. Oftentimes it creates something resembling a mixture of real images.

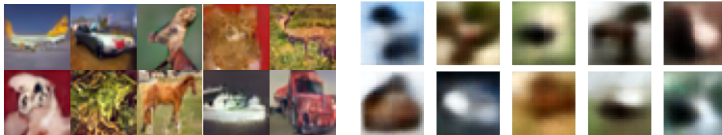


Figure 3: 10 generated images with the EMA model using the weight $w = 2$ to the left and the CVAE to the right. The EMA model generated the images with class conditioning, the classes starting from the top left are: aeroplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

In figure 4, the weight is set to $w = 5$ and the EMA model is compared to the regular model. Qualitatively, the regular model looks better and if this figure is compared to the images on the left part of figure 3, increasing w seems to yield higher quality. This showcases the effect of the guidance weight w , which is quite high in both figures. Increasing w might enforce the class conditioning more and therefore lead to higher quality samples, but, during testing, it could also result in less varied images. However, when comparing figure 3 to figure 4, it seems to mostly have improved the image quality. More examples are given in Appendix B.

The images generated from LFWPeople are shown in the two left-most sub-figures of figure 5. As can be observed qualitatively, the model without EMA and with EMA perform at a similar level. Typically EMA should perform better, and its results should therefore start to converge to higher

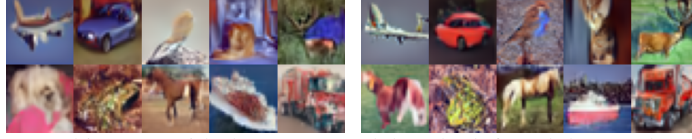


Figure 4: 10 generated images with the EMA model to the left and the regular DDPM to the right, both use the weight $w = 5$.

quality even more after further training is performed. Using this model as a starting point, 7 epochs were trained on the CelebA dataset. This is shown in the two right-most sub-figures of figure 5. A clear improvement is noticeable in these images.

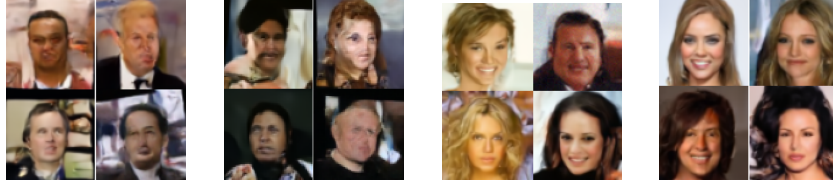


Figure 5: Four sub-figures with four unconditionally generated images each. The LFWpeople model without the EMA model is to the left and the EMA model is to the middle-left. The last two sub-figures, are 4 unconditionally generated images from the CelebA model pre-trained on LFWpeople, without EMA to the right and with EMA to the middle-right. Refer to Appendix B for more examples of generated faces.

The FID scores for all data sets are summarized in the table 1, where the FID score for CIFAR10 from [1] is included as a reference. Our values are much higher, indicating worse performance. However, they trained for 800k steps with a batch size of 128 while we trained for approximately 200k steps with a batch size of 24 so a performance difference is to be expected. However, the discrepancy seems very high and we are not sure exactly why. Further training and testing is necessary to determine this.

Model	CIFAR10	LFWPeople	CelebA
EMA	144	243	217
Reg	143	255	218
L_{simple} [1]	3.17	-	-

Table 1: FID Scores for our trained models on the MNIST, CIFAR10, LFWPeople, and CelebA (pre-trained on LFWPeople) Datasets, where Reg refers to the model with ordinary updates and EMA to the model with EMA updates. In the case of CIFAR10, the guidance weight was $w = 5$.

6 Conclusion

Our results clearly show that DDPMs are proficient models that can generate images with impressive quality. Our model, however, needs more training until it reaches an image quality comparable with the papers cited, which is especially clear given our high FID scores. Given the low number of epochs trained, it still seems to be a very capable and relevant architecture.

Another thing to note is that the EMA updated model takes longer to reach high quality, but is more stable and robust against too large model updates. If given more time, the EMA models would likely be the models obtaining the best quality and FID scores, but at the current level of training the models perform very similarly.

The main things that we learned in this project were how to implement a diffusion schedule, how to train a model with that schedule, and how to implement complex architectures with success, even if the FID scores are still relatively high. In the future, the model training could be continued to see the FID achieved at a later stage of the training.

References

1. Ho, J., Jain, A. & Abbeel, P. *Denoising Diffusion Probabilistic Models* 2020. arXiv: 2006.11239 [cs.LG].
2. Sohl-Dickstein, J., Weiss, E. A., Maheswaranathan, N. & Ganguli, S. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics* 2015. arXiv: 1503.03585 [cs.LG].
3. Bishop, C. M. & Bishop, H. *Deep Learning : Foundations and Concepts* ISBN: 9783031454677 (Cham, Switzerland, 2024).
4. Goncalves, A. *et al.* Generation and evaluation of synthetic patient data. *BMC medical research methodology* **20**, 1–40 (2020).
5. Nichol, A. & Dhariwal, P. *Improved Denoising Diffusion Probabilistic Models* 2021. arXiv: 2102.09672 [cs.LG].
6. Ho, J. & Salimans, T. *Classifier-Free Diffusion Guidance* 2022. arXiv: 2207.12598 [cs.LG].
7. Dhariwal, P. & Nichol, A. *Diffusion Models Beat GANs on Image Synthesis* 2021. arXiv: 2105.05233 [cs.LG].
8. Petit, O., Thome, N., Rambour, C. & Soler, L. *U-Net Transformer: Self and Cross Attention for Medical Image Segmentation* 2021. arXiv: 2103.06104 [eess.IV].
9. Rombach, R., Blattmann, A., Lorenz, D., Esser, P. & Ommer, B. *High-Resolution Image Synthesis with Latent Diffusion Models* 2022. arXiv: 2112.10752 [cs.CV].
10. Esser, P. *et al.* *Scaling Rectified Flow Transformers for High-Resolution Image Synthesis* 2024. arXiv: 2403.03206 [cs.CV].
11. Peebles, W. & Xie, S. *Scalable Diffusion Models with Transformers* 2023. arXiv: 2212.09748 [cs.CV].
12. Dosovitskiy, A. *et al.* *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* 2021. arXiv: 2010.11929 [cs.CV].
13. Deng, J. *et al.* ImageNet: A Large-Scale Hierarchical Image Database. *IEEE Computer Vision and Pattern Recognition (CVPR)*, 248–255. http://www.image-net.org/papers/imagenet_cvpr09.pdf (2009).
14. Salimans, T., Karpathy, A., Chen, X. & Kingma, D. P. PixelCNN++: Improving the Pixel-CNN with Discretized Logistic Mixture Likelihood and Other Modifications. *arXiv preprint arXiv:1701.05517*. arXiv: 1701.05517 [cs.LG]. <https://arxiv.org/pdf/1701.05517> (2017).
15. LeCun, Y., Cortes, C. & Burges, C. *The MNIST Database of Handwritten Digits* Accessed: 2024-05-21. 1998. <http://yann.lecun.com/exdb/mnist/>.
16. Krizhevsky, A. & Hinton, G. Learning Multiple Layers of Features from Tiny Images. *Technical Report, University of Toronto*. Accessed: 2024-05-21. <https://www.cs.toronto.edu/~kriz/cifar.html> (2009).
17. Huang, G. B., Mattar, M., Lee, H. & Learned-Miller, E. *Learning to align from scratch* <http://vis-www.cs.umass.edu/lfw/>. Available online. 2012.

18. Liu, Z., Luo, P., Wang, X. & Tang, X. *Deep Learning Face Attributes in the Wild* Accessed: 2024-05-21. 2015. <http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html>.
19. Sharma, A. A Deep Dive into Variational Autoencoders with PyTorch. *PyImageSearch*. <https://pyimagesearch.com/2023/10/02/a-deep-dive-into-variational-autoencoders-with-pytorch/> (2023).
20. Ronneberger, O., Fischer, P. & Brox, T. *U-Net: Convolutional Networks for Biomedical Image Segmentation* 2015. arXiv: 1505.04597 [cs.CV].
21. Wu, Y. & He, K. Group Normalization. *CoRR* **abs/1803.08494**. arXiv: 1803.08494. <http://arxiv.org/abs/1803.08494> (2018).
22. Vaswani, A. *et al.* Attention Is All You Need. *arXiv preprint arXiv:1706.03762* (2017).
23. DeepFindr. *A Diffusion Model from Scratch in Pytorch* <https://colab.research.google.com/drive/1sjy9odlSSy0RBVgMTgP7s99NXsqglsUL?usp=sharing#scrollTo=HhIgGq3za0yh>.
24. Dome272. *Diffusion-Models-pytorch* <https://github.com/dome272/Diffusion-Models-pytorch>. Accessed: 2024-05-12. 2022.
25. Paszke, A. *et al.* *PyTorch: An Imperative Style, High-Performance Deep Learning Library* 2019. arXiv: 1912.01703 [cs.LG].
26. Loshchilov, I. & Hutter, F. *Decoupled Weight Decay Regularization* 2019. arXiv: 1711.05101 [cs.LG].

A Mathematics Behind Diffusion Models

The contents of this appendix are primarily inspired by [3] and [1], and will cover the fundamental math needed to understand diffusion models. More in-depth explanations and derivations may be found in either of them.

An image from the training data is defined as $\mathbf{x}_0$. To this image, Gaussian noise is applied independently for each pixel to give noise-corrupted images $\mathbf{x}_1$ defined by

$$\mathbf{x}_1 = \sqrt{1 - \beta_1} \mathbf{x}_0 + \sqrt{\beta_1} \epsilon_1 \quad (1)$$

where $\epsilon_1 \sim \mathcal{N}(\epsilon_1|0, \mathbf{I})$ and β_1 is defined by the chosen noise schedule and determines the variance of the time step. This can be re-written in terms of the distribution

$$q(\mathbf{x}_1|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_1|\sqrt{1 - \beta_1} \mathbf{x}_0, \beta_1 \mathbf{I}). \quad (2)$$

This is then repeated for T time steps, and close to $t = T$ the image will be essentially pure noise. This requires that the schedule defining β_t for all t is chosen such that the mean of the next time step is closer to zero than the previous, while the variance moves closer to unit variance. By considering $q(\mathbf{x}_1|\mathbf{x}_0)$, it is clear that if β_t is chosen such that $\beta_{t-1} < \beta_t < 1 \forall t$, the requirements for the schedule will be met. By combining all time steps into one distribution, the approximate posterior

$$q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0) = \prod_{t=1}^T \mathcal{N}(\mathbf{x}_t|\sqrt{1 - \beta_t} \mathbf{x}_0, \beta_t \mathbf{I}) \quad (3)$$

is obtained, which, as is done in [3], can be written as

$$q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0) = q(\mathbf{x}_1|\mathbf{x}_0) \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}) \quad (4)$$

making the Markov chain interpretation clear. Considering the approximate posterior $q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0)$, the fixed encoder interpretation of a VAE is also clear since the goal of an encoder in a VAE is to learn the parameters defining the approximate posterior and these parameters are fixed here. In practice, any time step can be calculated directly from the noise-free image $\mathbf{x}_0$ with

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t|\sqrt{\alpha_t} \mathbf{x}_0, (1 - \alpha_t) \mathbf{I}) \quad (5)$$

where, again following the notation used in [3],

$$\alpha_t = \prod_{k=1}^t (1 - \beta_k). \quad (6)$$

During the sampling process, the time steps are in reverse. This means a Markov chain described by the transition probabilities $p(\mathbf{x}_{t-1}|\mathbf{x}_t)$. It is the parameters of these probabilities that the model needs to learn. By clever use of the reparameterization trick, it is possible to instead let the model learn to predict noise, rather than to directly predict the parameters of $p(\mathbf{x}_{t-1}|\mathbf{x}_t)$. Moreover, in this work, a fixed variance was used and set to $\sigma_t^2 = \beta_t$. This essentially means that only the mean will be determined by the predicted noise of the model. Following [1], the mean is given by

$$\mu = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta \right) \quad (7)$$

where $\epsilon_\theta = \epsilon_\theta(\mathbf{x}_t, t)$ and is the noise predicted by the model. The reparameterization trick then yields the sample $\mathbf{x}_{t-1}$ from $p(\mathbf{x}_{t-1}|\mathbf{x}_t)$ given by

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta \right) + \beta_t \mathbf{y} \quad (8)$$

where $\mathbf{y}$ is a sample from a standard Normal distribution. A simple sketch of both the forward and the reverse processes is shown in figure 6. With this approach, [1] shows that the loss to minimize is given by

$$\mathbb{E} \left[\frac{\beta_t^2}{2\beta_t(1 - \beta_t)(1 - \alpha_t)} \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] \quad (9)$$

where ϵ is a standard Gaussian sample from the forward pass used to calculate

$$\mathbf{x}_t = \sqrt{\alpha}\mathbf{x}_0 + \sqrt{1 - \alpha}\epsilon. \quad (10)$$

Then, as mentioned in section 4, it was found to be beneficial in [1] to simply use the loss

$$L = \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \quad (11)$$

meaning the factor in front was simply dropped. A thorough derivation starting from the KL-divergence can be found in [3].

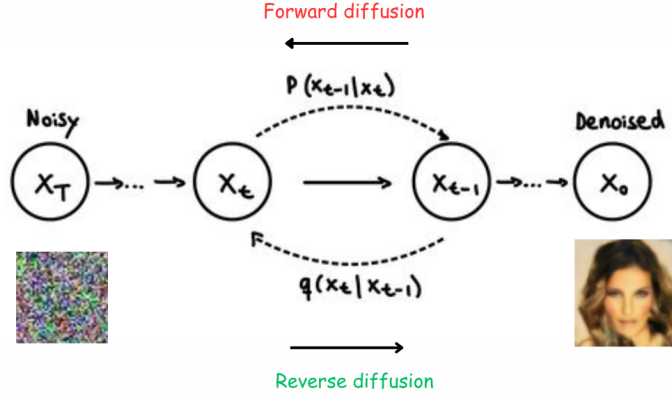


Figure 6: This figure shows a sketch of how the forward and reverse diffusion process takes place.

B Extra Images

In this appendix, extra generated images are shown. These are a mixture of images showing the diffusion process and simply more samples. In figure 7, the loss plot for the model trained on CIFAR10 for 100 epochs is shown.

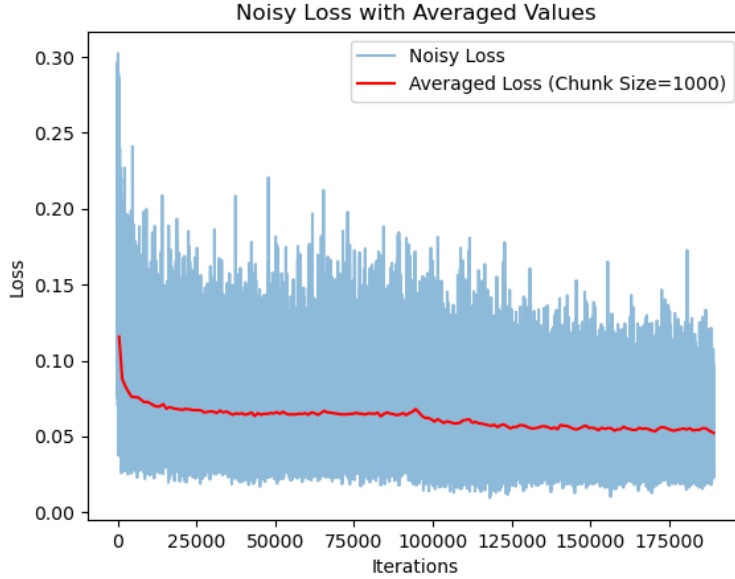


Figure 7: Loss plotted against the number of iterations for a model trained for 100 epochs.



Figure 8: Figure displaying the sampling process. The first image is pure noise and then every image is after 125 time steps. Left is EMA, right is DDPM and $w = 5.2$ in both cases and the models have been trained for 100 epochs.

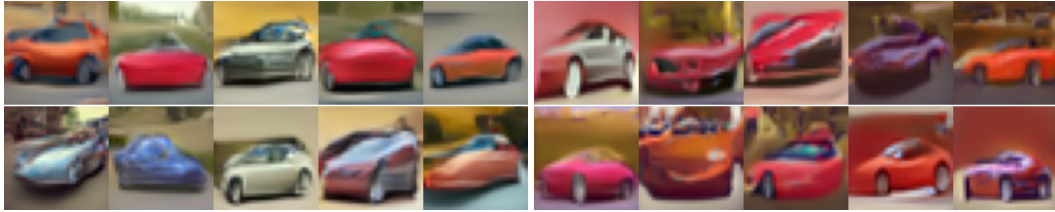


Figure 9: Left is EMA, right is DDPM and $w = 5$ in both cases and the models have been trained for 60 epochs. Models are conditioned to sample automobiles.

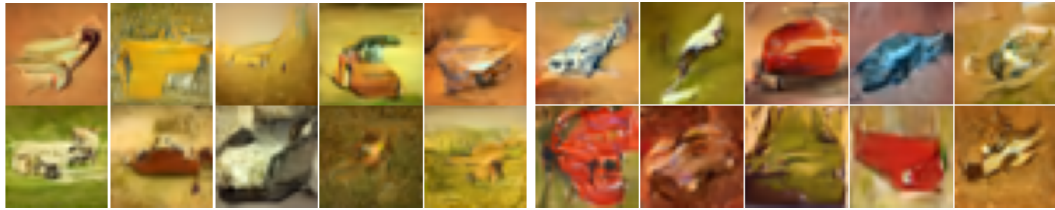


Figure 10: Left is EMA, right is DDPM and $w = 0$ in both cases and the models have been trained for 60 epochs. Models are conditioned to sample automobiles.

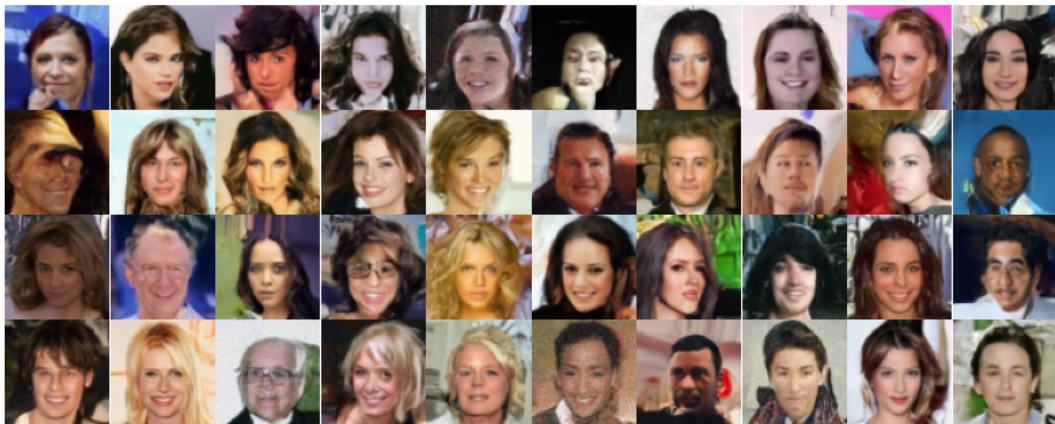


Figure 11: More sampled images of faces generated after 57 epochs of LFWpeople and 7 epochs on CelebA dataset with the EMA model.



Figure 12: More sampled images of faces generated after 57 epochs of LFWpeople and 7 epochs on CelebA dataset with the non-EMA model

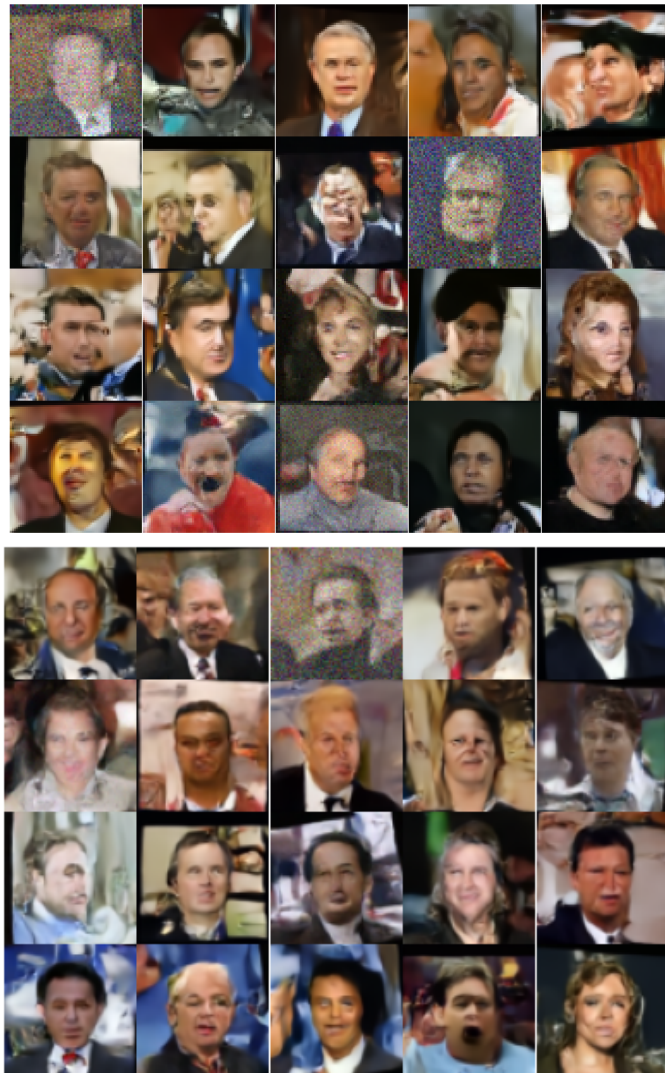


Figure 13: 20 generated images with the EMA model above and the non-EMA model below. This is for the model when trained on just LFWpeople.